

SpotMicro Jetson 로봇틱스

PyBullet API 마스터하기

Week 03 · 관절 제어와 순기구학 실습

stepSimulation · setJointMotorControl2 · PD 게인 · 순기구학(T0~T4)

저자 : robert

이름 : _____

목차

학습 목표	
사전 준비	
1. 시뮬레이션 기초	
2. 핵심 API	
3. PD 게인 튜닝	
4. 순기구학: 다리 관절 위치($T0 \sim T4$) 계산	
5. 실습: SpotMicro 제어	
6. 키보드로 SpotMicro 제어하기	
7. PyBullet GUI 슬라이더 사용법	
8. 과제	
9. 참고 자료	

PyBullet의 `stepSimulation`, `setJointMotorControl2` API를 마스터합니다. Position / Velocity / Torque 제어 모드를 실험하고, PD 게인 튜닝과 순기구학(Forward Kinematics)을 통한 다리 관절 위치 계산을 배웁니다.

학습 목표

이번 주차를 마치면 다음을 할 수 있습니다.

1. PyBullet 시뮬레이션의 기본 동작 원리를 이해한다.
2. 3가지 관절 제어 모드(Position / Velocity / Torque)를 구분해 사용한다.
3. PD 게인(Kp, Kd) 튜닝 방법을 익힌다.
4. 순기구학으로 다리 관절·발끝의 3차원 위치를 계산한다.
5. 중력 보상 토크를 계산하고 적용한다.

사전 준비

실습에는 다음 패키지가 필요합니다.

```
pip install pybullet numpy keyboard
```

참고: `keyboard` 라이브러리는 Windows에서 관리자 권한이 필요할 수 있습니다(7장 참조).

1. 시뮬레이션 기초

1.1 stepSimulation()

`stepSimulation()` 은 시뮬레이션을 한 타임스텝 진행시키는 함수입니다.

- 물리 엔진이 힘, 토크, 충돌 등을 계산합니다.
- 일반적으로 240Hz(1/240초) 간격으로 호출합니다.

```
import pybullet as p
import time

# 시뮬레이션 초기화
p.connect(p.GUI)
p.setGravity(0, 0, -9.81)

# 메인 루프
while True:
    p.stepSimulation()
    time.sleep(1./240.)
```

1.2 시뮬레이션 파라미터 설정

함수	역할
<code>p.setTimeStep()</code>	타임스텝 크기 설정
<code>p.setGravity()</code>	중력 설정
<code>p.setPhysicsEngineParameter()</code>	물리 엔진 세부 설정

2. 핵심 API

2.1 setJointMotorControl2() — 관절 제어

관절을 제어하는 핵심 함수로, 3가지 제어 모드를 지원합니다.

(1) POSITION_CONTROL (위치 제어)

- 목표 각도로 관절을 이동시킵니다.
- 내부적으로 PD 제어를 사용합니다.
- 가장 많이 사용되는 모드입니다.

```
p.setJointMotorControl2(
    bodyUniqueId=robotId,
    jointIndex=jointIndex,
    controlMode=p.POSITION_CONTROL,
    targetPosition=target_angle, # 목표 각도 (radians)
    force=maxForce,             # 최대 힘
    positionGain=Kp,             # P 게인
    velocityGain=Kd              # D 게인
)
```

(2) VELOCITY_CONTROL (속도 제어)

- 목표 속도로 관절을 회전시킵니다.
- 바퀴 구동 등에 적합합니다.

```
p.setJointMotorControl2(
    bodyUniqueId=robotId,
    jointIndex=jointIndex,
    controlMode=p.VELOCITY_CONTROL,
    targetVelocity=target_velocity, # 목표 속도 (rad/s)
    force=maxForce
)
```

(3) TORQUE_CONTROL (토크 제어)

- 직접 토크를 인가합니다.

- 가장 낮은 수준의 제어입니다.
- 중력 보상, 동역학 제어에 필수입니다.

```
p.setJointMotorControl2(
    bodyUniqueId=robotId,
    jointIndex=jointIndex,
    controlMode=p.TORQUE_CONTROL,
    force=torque # 인가할 토크
)
```

2.2 getJointState() — 관절 상태 읽기

관절의 현재 상태를 읽어옵니다.

```
joint_state = p.getJointState(robotId, jointIndex)
# 반환값: (position, velocity, reaction_forces, applied_torque)

position = joint_state[0]      # 현재 각도 (radians)
velocity = joint_state[1]      # 현재 각속도 (rad/s)
reaction = joint_state[2]      # 반력
applied_torque = joint_state[3] # 적용된 토크
```

2.3 getLinkState() — 링크 위치 읽기

링크(팔끝 등)의 위치와 자세를 계산합니다.

```
link_state = p.getLinkState(robotId, linkIndex)
# 반환값: (linkWorldPosition, linkWorldOrientation, ...)

position = link_state[0]      # 월드 좌표계에서의 위치 (x, y, z)
orientation = link_state[1]   # 월드 좌표계에서의 자세 (quaternion)
```

3. PD 게인 튜닝

3.1 PD 제어기의 원리

PD(Proportional-Derivative) 제어기는 위치 오차와 속도 오차를 이용하여 제어합니다.

$$\tau = K_p \times (\theta_{\text{target}} - \theta_{\text{current}}) + K_d \times (\omega_{\text{target}} - \omega_{\text{current}})$$

- **Kp (Position Gain):** 위치 오차에 대한 반응
 - 크면: 빠르게 목표에 도달하지만 진동 발생
 - 작으면: 느리게 도달하지만 안정적
- **Kd (Velocity Gain):** 속도 오차에 대한 반응(댐핑)
 - 크면: 진동을 빠르게 감소
 - 작으면: 진동이 오래 지속

3.2 튜닝 방법

1. **Kp부터 시작:** Kd=0으로 설정하고 Kp를 점진적으로 증가
2. **진동 관찰:** 목표 위치 주변에서 진동이 시작하는 지점 확인
3. **Kd 추가:** 진동을 감소시키기 위해 Kd를 증가
4. **미세 조정:** 성능과 안정성의 균형을 찾을 때까지 반복

```
# 예시: 게인 튜닝 실험
Kp_values = [0.1, 0.5, 1.0, 2.0, 5.0]
Kd_values = [0.01, 0.05, 0.1, 0.2, 0.5]

for Kp in Kp_values:
    for Kd in Kd_values:
        p.setJointMotorControl2(
            robotId, jointIndex,
            p.POSITION_CONTROL,
            targetPosition=target,
            positionGain=Kp,
            velocityGain=Kd
        )
```

PD 게인을 GUI 슬라이더로 실시간 조정하는 방법은 8장에서 다룹니다.

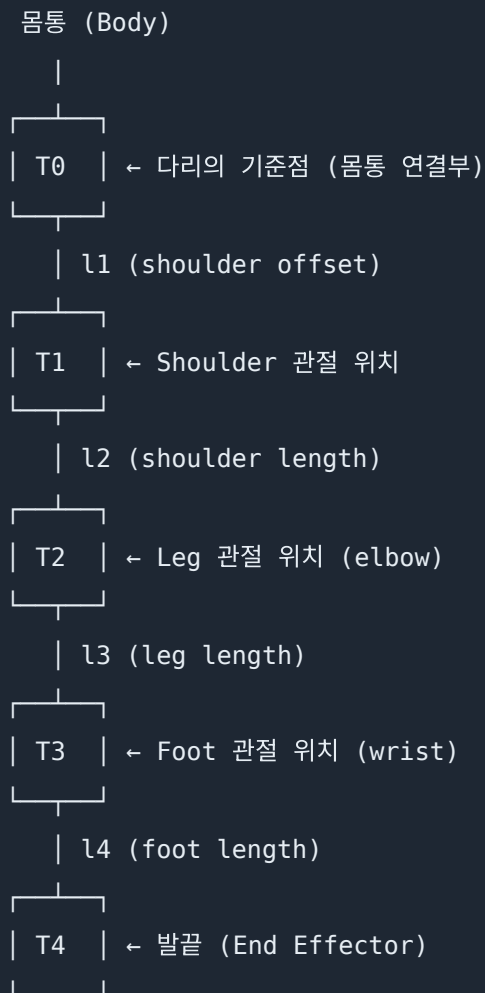
4. 순기구학: 다리 관절 위치(T0~T4) 계산

SpotMicro의 각 다리는 3개의 관절(shoulder, leg, foot)로 구성됩니다. 순기구학(Forward Kinematics)을 통해 각 관절과 발끝의 3차원 위치를 계산할 수 있습니다.

4.1 T0~T4의 의미

T0부터 T4는 다리의 기준점부터 발끝까지 각 관절의 위치를 나타냅니다.

[SpotMicro 다리 구조]



점	이름	의미	좌표계
T0	Base Point	다리의 기준점, 몸통과 연결되는 지점	(0, 0, 0)
T1	Shoulder Joint	첫 번째 관절 (좌우 회전)	T0 기준 상대 위치
T2	Leg Joint	두 번째 관절 (상하 회전, elbow)	T1 기준 상대 위치
T3	Foot Joint	세 번째 관절 (상하 회전, wrist)	T2 기준 상대 위치
T4	End Effector	발끝, 지면과 접촉하는 지점	T3 기준 상대 위치

4.2 수학적 정의

관절 각도 $(\theta_1, \theta_2, \theta_3)$ 가 주어졌을 때, 각 위치는 다음과 같이 계산됩니다.

```

# kinematics.py의 calcLegPoints 함수
def calcLegPoints(self, angles):
    (l1, l2, l3, l4) = (self.l1, self.l2, self.l3, self.l4)
    (theta1, theta2, theta3) = angles
    theta23 = theta2 + theta3

    # T0: 기준점 (몸통 연결부)
    T0 = np.array([0, 0, 0, 1])

    # T1: Shoulder 관절
    T1 = T0 + np.array([-l1*cos(theta1), l1*sin(theta1), 0, 0])

    # T2: Leg 관절 (elbow)
    T2 = T1 + np.array([-l2*sin(theta1), -l2*cos(theta1), 0, 0])

    # T3: Foot 관절 (wrist)
    T3 = T2 + np.array([
        -l3*sin(theta1)*cos(theta2),
        -l3*cos(theta1)*cos(theta2),
        l3*sin(theta2),
        0
    ])

    # T4: 발끝 (End Effector)
    T4 = T3 + np.array([
        -l4*sin(theta1)*cos(theta23),
        -l4*cos(theta1)*cos(theta23),
        l4*sin(theta23),
        0
    ])

    return np.array([T0, T1, T2, T3, T4])

```

4.3 링크 길이와 관절 각도

링크 길이 (Link Lengths) — SpotMicro의 다리 링크 길이:

```
# 단위: mm
l1 = 50    # Shoulder offset (몸통에서 shoulder 관절까지)
l2 = 20    # Shoulder length (shoulder에서 leg 관절까지)
l3 = 100   # Leg length (leg에서 foot 관절까지, 상완)
l4 = 115   # Foot length (foot 관절에서 발끝까지, 하완)
```

관절 각도 (Joint Angles):

```
theta1 (θ1): Shoulder 관절 각도 (좌우 회전, abduction/adduction)
theta2 (θ2): Leg 관절 각도 (상하 회전, flexion/extension)
theta3 (θ3): Foot 관절 각도 (상하 회전, flexion/extension)
```

4.4 실습 예제: 다리 관절 위치 계산

```
import numpy as np
import pybullet as p
from math import cos, sin

class LegKinematics:
    def __init__(self):
        # 링크 길이 (mm)
        self.l1 = 50
        self.l2 = 20
        self.l3 = 100
        self.l4 = 115

    def calc_leg_points(self, theta1, theta2, theta3):
        """관절 각도로부터 각 관절의 위치 계산"""
        l1, l2, l3, l4 = self.l1, self.l2, self.l3, self.l4
        theta23 = theta2 + theta3

        # 각 관절 위치 계산
        T0 = np.array([0, 0, 0, 1])
        T1 = T0 + np.array([-l1*cos(theta1), l1*sin(theta1), 0, 0])
        T2 = T1 + np.array([-l2*sin(theta1), -l2*cos(theta1), 0, 0])
        T3 = T2 + np.array([
            -l3*sin(theta1)*cos(theta2),
            -l3*cos(theta1)*cos(theta2),
            l3*sin(theta2),
            0
        ])
        T4 = T3 + np.array([
            -l4*sin(theta1)*cos(theta23),
            -l4*cos(theta1)*cos(theta23),
            l4*sin(theta23),
            0
        ])

        return T0, T1, T2, T3, T4

# 사용 예시
leg_kin = LegKinematics()
```

```

# Home position 각도 (radians)
theta1 = 0.0          # shoulder: 중립
theta2 = -0.8         # leg: 약간 구부림
theta3 = 1.6          # foot: 많이 구부림

# 각 관절 위치 계산
T0, T1, T2, T3, T4 = leg_kin.calc_leg_points(theta1, theta2, theta3)

print("=== 다리 관절 위치 ===")
print(f"T0 (Base):      x={T0[0]:6.1f}, y={T0[1]:6.1f}, z={T0[2]:6.1f} mm")
print(f"T1 (Shoulder):  x={T1[0]:6.1f}, y={T1[1]:6.1f}, z={T1[2]:6.1f} mm")
print(f"T2 (Leg):         x={T2[0]:6.1f}, y={T2[1]:6.1f}, z={T2[2]:6.1f} mm")
print(f"T3 (Foot):       x={T3[0]:6.1f}, y={T3[1]:6.1f}, z={T3[2]:6.1f} mm")
print(f"T4 (End):        x={T4[0]:6.1f}, y={T4[1]:6.1f}, z={T4[2]:6.1f} mm")

```

4.5 PyBullet에서 관절 위치 시각화

```
import pybullet as p
import time

# PyBullet 초기화
p.connect(p.GUI)
robot = p.loadURDF("spot_micro.urdf", [0, 0, 0.3])

# 각 다리의 관절 인덱스
leg_joints = {
    'front_left': [0, 1, 2],    # shoulder, leg, foot
    'front_right': [3, 4, 5],
    'rear_left': [6, 7, 8],
    'rear_right': [9, 10, 11]
}

# 관절 각도 설정
target_angles = [0.0, -0.8, 1.6] # shoulder, leg, foot

while True:
    for leg_name, joints in leg_joints.items():
        # 각 관절에 각도 적용
        for joint_idx, angle in zip(joints, target_angles):
            p.setJointMotorControl2(
                robot,
                joint_idx,
                p.POSITION_CONTROL,
                targetPosition=angle
            )

        # 각 관절의 현재 위치 읽기 (PyBullet의 getLinkState)
        for i, joint_idx in enumerate(joints):
            link_state = p.getLinkState(robot, joint_idx)
            pos = link_state[0] # 월드 좌표계 위치

            joint_names = ['Shoulder (T1)', 'Leg (T2)', 'Foot (T3)']
            print(f"{leg_name} {joint_names[i]}: "
                  f"x={pos[0]*1000:.1f}, y={pos[1]*1000:.1f}, z={pos[2]*1000:.1f} mm")
```

```

# 발끝 위치 (T4)
foot_link = joints[2] + 1 # foot 관절 다음 링크
link_state = p.getLinkState(robot, foot_link)
pos = link_state[0]
print(f"{leg_name} End (T4): "
      f"x={pos[0]*1000:.1f}, y={pos[1]*1000:.1f}, z={pos[2]*1000:.1f} mm")
print()

p.stepSimulation()
time.sleep(1./240.)

```

4.6 좌표계 변환

각 T는 **다리 로컬 좌표계**에서의 위치입니다. 월드 좌표계로 변환하려면 다음과 같이 합니다.

```

# 다리의 월드 좌표계 변환 행렬
T_world_to_leg = np.array([
    [cos(yaw), -sin(yaw), 0, body_x],
    [sin(yaw),  cos(yaw), 0, body_y],
    [0,          0,          1, body_z],
    [0,          0,          0,  1]
])

# 로컬 좌표 → 월드 좌표
T4_world = T_world_to_leg.dot(T4)

```

4.7 T0~T4의 활용

1. 순기구학 (Forward Kinematics)

- 입력: 관절 각도 ($\theta_1, \theta_2, \theta_3$) → 출력: 발끝 위치 (T4)
- 용도: 관절 각도가 주어졌을 때 발끝이 어디에 있는지 계산

2. 충돌 검사

- T1, T2, T3, T4를 연결한 선분으로 다리 형상 표현
- 다리가 몸통이나 다른 다리와 충돌하는지 확인

3. 시각화

- 디버깅 시 각 관절의 위치를 화면에 표시해 다리의 움직임을 이해

4. 작업 공간 (Workspace) 계산

- 발끝(T4)이 도달할 수 있는 모든 위치의 범위
- 보행 계획 시 발을 놓을 수 있는 위치 결정

정리 - T0~T4: 다리의 각 관절 및 발끝의 3차원 위치 - **순기구학:** 관절 각도 → 발끝 위치 계산 (이번 장) - **역기구학(Inverse Kinematics):** 발끝 위치 → 관절 각도 계산 (과제 4번에서 활용, 다음 주차에서 본격적으로 다룸)

5. 실습: SpotMicro 제어

5.1 Home Position 제어

SpotMicro를 서 있는 자세로 만듭니다.

```

import pybullet as p
import pybullet_data
import time

# 초기화
p.connect(p.GUI)
p.setAdditionalSearchPath(pybullet_data.getDataPath())
p.setGravity(0, 0, -9.81)
plane = p.loadURDF("plane.urdf")
robot = p.loadURDF("spot_micro.urdf", [0, 0, 0.3])

# Home position 각도 정의 (예시)
home_positions = {
    'front_left_leg': [0.0, -0.8, 1.6], # shoulder, elbow, wrist
    'front_right_leg': [0.0, -0.8, 1.6],
    'rear_left_leg': [0.0, -0.8, 1.6],
    'rear_right_leg': [0.0, -0.8, 1.6]
}

# 관절 인덱스 얻기
num_joints = p.getNumJoints(robot)
joint_indices = {}
for i in range(num_joints):
    joint_info = p.getJointInfo(robot, i)
    joint_name = joint_info[1].decode('utf-8')
    joint_indices[joint_name] = i

# PD 게인 설정
Kp = 1.0
Kd = 0.1

# 제어 루프
for _ in range(10000):
    # 모든 관절을 home position으로 제어
    for leg, angles in home_positions.items():
        for i, angle in enumerate(angles):
            joint_name = f"{leg}_joint_{i}"
            if joint_name in joint_indices:
                p.setJointMotorControl2(

```

```
        robot,\n        joint_indices[joint_name],\n        p.POSITION_CONTROL,\n        targetPosition=angle,\n        force=50,\n        positionGain=Kp,\n        velocityGain=Kd\n    )\n\n    p.stepSimulation()\n    time.sleep(1./240.)\n\np.disconnect()
```

5.2 중력 보상 토크 계산

중력에 의한 관절 토크를 역동역학(Inverse Dynamics)으로 계산하고 보상합니다.

```

# 1. 역동역학을 이용한 방법
def calculate_gravity_compensation(robot, joint_positions):
    """
    현재 자세에서 중력 보상 토크 계산
    """
    num_joints = p.getNumJoints(robot)

    # 영속도, 영가속도 상태에서의 토크 계산
    zero_velocities = [0.0] * num_joints
    zero_accelerations = [0.0] * num_joints

    gravity_compensation = p.calculateInverseDynamics(
        robot,
        joint_positions,
        zero_velocities,
        zero_accelerations
    )

    return gravity_compensation

# 2. 중력 보상 토크 적용
while True:
    # 현재 관절 각도 읽기
    joint_positions = []
    for i in range(p.getNumJoints(robot)):
        joint_state = p.getJointState(robot, i)
        joint_positions.append(joint_state[0])

    # 중력 보상 토크 계산
    gravity_torques = calculate_gravity_compensation(robot, joint_positions)

    # 토크 제어 모드로 적용
    for i in range(p.getNumJoints(robot)):
        p.setJointMotorControl2(
            robot, i,
            p.TORQUE_CONTROL,
            force=gravity_torques[i]
        )

```

```
p.stepSimulation()
time.sleep(1./240.)
```

5.3 발끝 위치 확인

`getLinkState()` 로 각 발끝 링크의 월드 좌표를 읽어 확인합니다. (이론은 4장 순기구학 참조)

```
# 발끝 링크 인덱스 (URDF에 따라 다름)
foot_links = {
    'front_left': 3,
    'front_right': 7,
    'rear_left': 11,
    'rear_right': 15
}

# 발끝 위치 출력
for foot_name, link_index in foot_links.items():
    link_state = p.getLinkState(robot, link_index)
    position = link_state[0]
    print(f"{foot_name} foot position: x={position[0]:.3f}, y={position[1]:.3f}, z={position[2]:.3f}")
```

6. 키보드로 SpotMicro 제어하기

PyBullet에서 키보드 입력을 받아 로봇을 제어하는 세 가지 방법을 알아봅니다.

6.1 방법 1: addUserDebugParameter() — GUI 슬라이더

GUI 슬라이더를 통해 실시간으로 값을 조정하는 방식입니다.

```
import pybullet as p
import time

# 초기화
p.connect(p.GUI)
robot = p.loadURDF("spot_micro.urdf", [0, 0, 0.3])

# 디버그 파라미터 생성 (GUI 슬라이더)
x_slider = p.addUserDebugParameter("X Position", -1.0, 1.0, 0.0)
y_slider = p.addUserDebugParameter("Y Position", -1.0, 1.0, 0.0)
z_slider = p.addUserDebugParameter("Z Position", -0.5, 0.5, 0.0)
yaw_slider = p.addUserDebugParameter("Yaw", -3.14, 3.14, 0.0)

# 메인 루프
while True:
    # 슬라이더 값 읽기
    x_pos = p.readUserDebugParameter(x_slider)
    y_pos = p.readUserDebugParameter(y_slider)
    z_pos = p.readUserDebugParameter(z_slider)
    yaw = p.readUserDebugParameter(yaw_slider)

    # 로봇 제어에 사용
    # ... 역기구학 계산 및 관절 제어

    p.stepSimulation()
    time.sleep(1./240.)
```

- **장점:** 간단하고 실시간 조정이 쉬움
- **단점:** 슬라이더로만 조작 가능, 정밀한 키보드 제어 불가

6.2 방법 2: keyboard 라이브러리

`keyboard` 라이브러리를 사용하여 키 입력을 직접 처리합니다.


```

import pybullet as p
import pybullet_data
import time
import keyboard

# 초기화
p.connect(p.GUI)
p.setAdditionalSearchPath(pybullet_data.getDataPath())
p.setGravity(0, 0, -9.81)
plane = p.loadURDF("plane.urdf")
robot = p.loadURDF("spot_micro.urdf", [0, 0, 0.3])

# 제어 변수
velocity_x = 0.0
velocity_y = 0.0
velocity_yaw = 0.0
height_offset = 0.0

SPEED = 0.1 # 이동 속도
YAW_SPEED = 0.05 # 회전 속도
HEIGHT_SPEED = 0.01 # 높이 변화 속도

# 메인 루프
while True:
    # 키보드 입력 처리
    if keyboard.is_pressed('w'):
        velocity_x = SPEED
    elif keyboard.is_pressed('s'):
        velocity_x = -SPEED
    else:
        velocity_x = 0.0

    if keyboard.is_pressed('a'):
        velocity_y = SPEED
    elif keyboard.is_pressed('d'):
        velocity_y = -SPEED
    else:
        velocity_y = 0.0

```

```

if keyboard.is_pressed('q'):
    velocity_yaw = YAW_SPEED
elif keyboard.is_pressed('e'):
    velocity_yaw = -YAW_SPEED
else:
    velocity_yaw = 0.0

if keyboard.is_pressed('up'):
    height_offset += HEIGHT_SPEED
elif keyboard.is_pressed('down'):
    height_offset -= HEIGHT_SPEED

# ESC로 종료
if keyboard.is_pressed('esc'):
    break

# 속도를 이용하여 로봇 제어
# velocity_x, velocity_y, velocity_yaw를 보행 패턴에 전달

p.stepSimulation()
time.sleep(1./240.)

p.disconnect()

```

설치 방법:

```
pip install keyboard
```

주의: Windows에서는 관리자 권한이 필요할 수 있습니다.

6.3 방법 3: 멀티프로세싱 (실제 프로젝트 방식)

SpotMicroAI 프로젝트에서 사용하는 방식으로, 키보드 입력을 별도 프로세스에서 처리합니다.

```

import time
import keyboard
from multiprocessing import Process, Queue

# 키보드 입력 기본값
key_value_default = {'w': 0, 'a': 0, 's': 0, 'd': 0, 'q': 0, 'e': 0, 'move': False}
control_offset = {'IDstepLength': 0.0, 'IDstepWidth': 0.0, 'IDstepAlpha': 0.0, 'Star

class KeyboardController:

    def __init__(self):
        # Queue를 사용하여 프로세스 간 데이터 공유
        self.key_status = Queue()
        self.key_status.put(key_value_default)

        self.command_status = Queue()
        self.command_status.put(control_offset)

        # 제어 파라미터
        self.X_STEP = 10.0 # 전후 이동 보폭
        self.Y_STEP = 5.0 # 좌우 이동 보폭
        self.YAW_STEP = 3.0 # 회전 각도

    def resetStatus(self):
        """스페이스바로 상태 리셋"""
        result_dict = self.key_status.get()
        self.key_status.put(key_value_default)

    def keyCounter(self, character):
        """키가 눌렸을 때 카운터 증가"""
        result_dict = self.key_status.get()
        result_dict[character] += 1
        result_dict['move'] = True
        self.key_status.put(result_dict)

    def calcRobotStep(self):
        """키 입력을 로봇 제어 명령으로 변환"""
        result_dict = self.key_status.get()
        command_dict = self.command_status.get()

```

```

# 전후 이동 (W/S 키)
command_dict['IDstepLength'] = self.X_STEP * result_dict['s'] - self.X_STEP

# 좌우 이동 (A/D 키)
command_dict['IDstepWidth'] = self.Y_STEP * result_dict['d'] - self.Y_STEP *

# 회전 (Q/E 키)
command_dict['IDstepAlpha'] = self.YAW_STEP * result_dict['q'] - self.YAW_ST

# 걷기 시작/정지
if result_dict['move']:
    command_dict['StartStepping'] = True
else:
    command_dict['StartStepping'] = False

self.key_status.put(result_dict)
self.command_status.put(command_dict)

def keyboardProcess(self, id, key_status, command_status):
    """키보드 입력을 처리하는 프로세스"""
    was_pressed = False

    while True:
        if keyboard.is_pressed('w'):
            if not was_pressed:
                self.keyCounter('w')
                was_pressed = True
        elif keyboard.is_pressed('a'):
            if not was_pressed:
                self.keyCounter('a')
                was_pressed = True
        elif keyboard.is_pressed('s'):
            if not was_pressed:
                self.keyCounter('s')
                was_pressed = True
        elif keyboard.is_pressed('d'):
            if not was_pressed:
                self.keyCounter('d')
                was_pressed = True

```

```

        elif keyboard.is_pressed('q'):
            if not was_pressed:
                self.keyCounter('q')
                was_pressed = True
        elif keyboard.is_pressed('e'):
            if not was_pressed:
                self.keyCounter('e')
                was_pressed = True
        elif keyboard.is_pressed('space'):
            if not was_pressed:
                self.resetStatus()
                was_pressed = True
        else:
            was_pressed = False

        self.calcRobotStep()
        time.sleep(0.01)  # CPU 사용률 감소

def main(id, command_status):
    """메인 시뮬레이션 루프"""
    import pybullet as p
    import pybullet_data

    # PyBullet 초기화 (메인 프로세스에서만)
    p.connect(p.GUI)
    p.setAdditionalSearchPath(pybullet_data.getDataPath())
    p.setGravity(0, 0, -9.81)
    plane = p.loadURDF("plane.urdf")
    robot = p.loadURDF("spot_micro.urdf", [0, 0, 0.3])

    while True:
        # 키보드 명령 읽기
        result_dict = command_status.get()
        print(f"Step Length: {result_dict['IDstepLength']}, "
              f"Step Width: {result_dict['IDstepWidth']}, "
              f"Yaw: {result_dict['IDstepAlpha']}, "
              f"Walking: {result_dict['StartStepping']}")
        command_status.put(result_dict)

```

```

    # 로봇 제어
    if result_dict['StartStepping']:
        # 보행 패턴 실행
        pass
    else:
        # 정지 자세 유지
        pass

    p.stepSimulation()
    time.sleep(1./240.)

if __name__ == "__main__":
    try:
        # 키보드 입력 프로세스 생성
        KeyInputs = KeyboardController()
        KeyProcess = Process(target=KeyInputs.keyboardProcess,
                             args=(1, KeyInputs.key_status, KeyInputs.command_status))
        KeyProcess.start()

        # 메인 시뮬레이션 프로세스
        main(2, KeyInputs.command_status)

    except Exception as e:
        print(e)
    finally:
        # 종료 시 키보드 프로세스 정리
        if KeyProcess.is_alive():
            KeyProcess.terminate()
        print("Done!")

```

6.4 키 �핑 (Key Mapping)

키	기능
W	전진
S	후진
A	좌측 이동
D	우측 이동
Q	좌회전 (반시계)
E	우회전 (시계)
Space	리셋 (정지)
↑	높이 증가 (옵션)
↓	높이 감소 (옵션)
ESC	종료 (옵션)

6.5 방법 비교

방법	장점	단점	추천 용도
Debug Parameter	간단, 실시간 조정	슬라이더만 가능	파라미터 튜닝
keyboard 라이브러리	구현 간단	관리자 권한 필요	간단한 제어
멀티프로세싱	안정적, 확장 가능	복잡한 구조	실제 프로젝트

주의사항

1. **프로세스 관리**: 멀티프로세싱 사용 시 반드시 자식 프로세스를 정리해야 함
2. **GUI 중복**: Robot 객체는 메인 프로세스에서만 생성 (방법 3 참조)
3. **키보드 권한**: Windows에서는 관리자 권한이 필요할 수 있음
4. **동기화**: Queue를 사용하여 프로세스 간 데이터 안전하게 공유

6.6 키보드 입력 처리 원리 (상세)

1. 키 눌림 감지 메커니즘

키보드 입력을 처리할 때는 **키가 눌린 순간**과 **키가 계속 눌러있는 상태**를 구분해야 합니다.

```

was_pressed = False # 이전 상태 저장

while True:
    if keyboard.is_pressed('w'):
        if not was_pressed: # 이번에 처음 눌렀을 때만
            print("W 키가 눌렸습니다!")
            # 한 번만 실행되어야 하는 동작
            was_pressed = True
        # 키가 계속 눌러있는 동안 실행되는 동작
    else:
        was_pressed = False # 키를 떼면 상태 초기화

```

왜 이렇게 해야 하나요?

- `keyboard.is_pressed()` 는 키가 눌러있는 동안 계속 `True` 를 반환
- `was_pressed` 플래그 없이 사용하면 한 번 누르는 동안 여러 번 실행됨
- 로봇 제어에서는 "한 번 눌렀을 때 1 증가" 같은 동작이 필요

2. 키 카운터 방식의 이해

SpotMicroAI 프로젝트에서 사용하는 키 카운터 방식:

```

# 키 상태 딕셔너리
key_status = {'w': 0, 's': 0, 'a': 0, 'd': 0, 'move': False}

# W 키를 누르면 카운터 증가
if keyboard.is_pressed('w'):
    if not was_pressed:
        key_status['w'] += 1 # 0 -> 1 -> 2 -> 3 ...
        key_status['move'] = True
        was_pressed = True

```

동작 원리:

1. 각 키마다 카운터가 있음 (w, s, a, d, q, e)
2. 키를 누를 때마다 해당 카운터가 +1 증가
3. 이 카운터 값을 로봇 제어 명령으로 변환

왜 카운터를 사용하나요?


```
# W를 3번, S를 1번 눌렀다면
step_length = X_STEP * key_status['s'] - X_STEP * key_status['w']
              = 10.0 * 1 - 10.0 * 3
              = -20.0 # 앞으로 20mm 이동
```

- 누적된 입력을 반영 (W 3번 = 더 많이 전진)
- Space 키로 리셋 가능

3. Queue를 사용한 프로세스 간 통신

멀티프로세싱에서는 프로세스끼리 직접 변수를 공유할 수 없습니다. **Queue** 를 사용해야 합니다.

```
from multiprocessing import Queue

# Queue 생성
command_status = Queue()

# 프로세스 1: 데이터 넣기
command_status.put({'speed': 10, 'direction': 'forward'})

# 프로세스 2: 데이터 꺼내기
data = command_status.get() # {'speed': 10, 'direction': 'forward'}
```

Queue의 get/put 패턴:

```

# 키보드 프로세스
def keyboard_process(command_status):
    while True:
        if keyboard.is_pressed('w'):
            # 1. Queue에서 현재 데이터 가져오기
            result = command_status.get()

            # 2. 데이터 수정
            result['speed'] += 1

            # 3. 다시 Queue에 넣기
            command_status.put(result)

# 메인 프로세스
def main(command_status):
    while True:
        # 1. Queue에서 데이터 가져오기
        result = command_status.get()

        # 2. 데이터 사용 (로봇 제어)
        print(f"Speed: {result['speed']}")

        # 3. 다시 Queue에 넣기 (중요!)
        command_status.put(result)

```

중요 포인트:

- `get()` 으로 꺼내면 Queue가 비어짐
- 사용 후 반드시 `put()` 으로 다시 넣어야 함
- 안 넣으면 다른 프로세스가 `get()` 에서 멈춤 (블로킹)

4. 데이터 흐름 다이어그램



5. 실제 동작 예시

사용자가 **W** 키를 2번, **D** 키를 1번 누른 경우:

```
# 초기 상태
key_status = {'w': 0, 's': 0, 'a': 0, 'd': 0}

# W 첫 번째 누름
key_status = {'w': 1, 's': 0, 'a': 0, 'd': 0, 'move': True}

# W 두 번째 누름
key_status = {'w': 2, 's': 0, 'a': 0, 'd': 0, 'move': True}

# D 누름
key_status = {'w': 2, 's': 0, 'a': 0, 'd': 1, 'move': True}

# calcRobotStep() 실행 결과
command_status = {
    'IDstepLength': -20.0, #  $10 * 0 - 10 * 2 = -20$  (전진)
    'IDstepWidth': -5.0, #  $5 * 1 - 5 * 0 = 5$  (우측)
    'IDstepAlpha': 0.0, #  $3 * 0 - 3 * 0 = 0$  (회전 없음)
    'StartStepping': True # 걷기 시작
}
```

Space 키를 누르면:

```
# 모든 카운터 초기화
key_status = {'w': 0, 's': 0, 'a': 0, 'd': 0, 'move': False}

command_status = {
    'IDstepLength': 0.0,
    'IDstepWidth': 0.0,
    'IDstepAlpha': 0.0,
    'StartStepping': False # 걷기 정지
}
```

6. 디버깅 팁

키 입력이 안 될 때:

```
# 키보드 프로세스에서 디버그 출력
def keyboardProcess(self, id, key_status, command_status):
    while True:
        if keyboard.is_pressed('w'):
            print("W 키 감지됨!") # 추가
        if not was_pressed:
            self.keyCounter('w')
            print(f"카운터 증가: {key_status.get()}") # 추가
```

Queue가 막혔을 때:

```
# 타임아웃 설정
try:
    result = command_status.get(timeout=1) # 1초 대기
except:
    print("Queue에서 데이터를 받지 못했습니다!")
```

프로세스 종료 안 될 때:

```
# 강제 종료
KeyProcess.terminate() # SIGTERM 전송
KeyProcess.join(timeout=2) # 2초 대기
if KeyProcess.is_alive():
    KeyProcess.kill() # SIGKILL 전송 (강제)
```

7. 성능 최적화

```
def keyboardProcess(self):
    while True:
        # ... 키 처리 ...

        time.sleep(0.01) # 중요! CPU 사용을 감소
```

왜 sleep이 필요한가요?

- `while True` 무한루프는 CPU를 100% 사용
- `time.sleep(0.01)` : 10ms마다 한 번씩 체크
- 사람이 느끼기에는 즉각적이지만 CPU는 99% 씬

- 키보드 입력은 10ms 지연이 문제 안 됨

7. PyBullet GUI 슬라이더 사용법

PyBullet GUI에서 제공하는 슬라이더는 실시간으로 파라미터를 조정할 수 있는 강력한 도구입니다.

7.1 주요 슬라이더의 종류와 기능

SpotMicroAI 프로젝트에서 사용하는 주요 슬라이더들입니다.

1. Kp (Position Gain) 슬라이더

```
IDkp = p.addUserDebugParameter("Kp", 0, 0.05, 0.012)
```

- 위치: GUI 왼쪽 상단
- 범위: 0.0 ~ 0.05
- 기본값: 0.012
- 용도: PD 제어기의 비례 게인 조정

실시간 읽기:

```
kp = p.readUserDebugParameter(IDkp)
```

효과:

- 값을 증가시키면: 관절이 목표 위치에 더 빠르게 도달하지만 진동 발생 가능
- 값을 감소시키면: 움직임이 느려지지만 더 안정적

실험 방법:

1. 시뮬레이션을 실행
2. Kp 슬라이더를 천천히 오른쪽으로 이동 (증가)
3. 로봇 다리의 반응 속도가 빨라지는 것 관찰
4. 너무 높이면 진동/떨림 발생 → 최적값 찾기

2. Kd (Velocity Gain) 슬라이더

```
IDkd = p.addUserDebugParameter("Kd", 0, 1, 0.4)
```

- 위치: GUI 왼쪽 상단 (Kp 아래)

- 범위: 0.0 ~ 1.0
- 기본값: 0.4
- 용도: PD 제어기의 미분 게인 조정 (댐핑)

효과:

- 값을 증가시키면: 진동이 빠르게 감소, 움직임이 부드러움
- 값을 감소시키면: 진동이 오래 지속, 움직임이 덜 부드러움

실험 방법:

1. Kp를 높여서 진동이 발생하도록 설정
2. Kd 슬라이더를 증가시키면서 진동이 줄어드는 것 관찰
3. 최적의 Kd 값 찾기 (진동 제거 + 빠른 응답)

3. MaxForce 슬라이더

```
IDmaxForce = p.addUserDebugParameter("MaxForce", 0, 50, 12.5)
```

- 위치: GUI 왼쪽 상단 (Kd 아래)
- 범위: 0.0 ~ 50.0
- 기본값: 12.5
- 용도: 관절 모터에 가할 수 있는 최대 힘 제한

효과:

- 값을 증가시키면: 더 강한 힘으로 움직임, 무거운 물체를 들 수 있음
- 값을 감소시키면: 약한 힘, 중력에 의해 쉽게 처짐

실험 방법:

1. MaxForce를 0에 가깝게 설정 → 로봇이 중력에 의해 쓰러짐
2. 점진적으로 증가 → 로봇이 자세를 유지하는 최소값 찾기
3. 너무 높게 설정 → 에너지 낭비, 비현실적인 움직임

4. height 슬라이더

```
IDheight = p.addUserDebugParameter("height", -40, 90, 20)
```

- 위치: GUI 왼쪽 상단
- 범위: -40 ~ 90 (mm)

- **기본값:** 20
- **용도:** 로봇 몸통의 높이 조정

효과:

- **값을 증가**시키면: 로봇이 높이 올라감 (다리가 펴짐)
- **값을 감소**시키면: 로봇이 낮아짐 (다리가 구부러짐)

실험 방법:

1. 슬라이더를 최소값(-40)으로 → 로봇이 웅크림
2. 슬라이더를 최대값(90)으로 → 로봇이 최대한 펴짐
3. 걷기 동작 중 높이 변경 → 보행 패턴 변화 관찰

7.2 슬라이더 사용 패턴

```
import pybullet as p

# 1. 슬라이더 생성 (한 번만)
IDkp = p.addUserDebugParameter("Kp", 0, 0.05, 0.012)
IDkd = p.addUserDebugParameter("Kd", 0, 1, 0.4)
IDmaxForce = p.addUserDebugParameter("MaxForce", 0, 50, 12.5)
IDheight = p.addUserDebugParameter("height", -40, 90, 20)

# 2. 메인 루프에서 값 읽기 (매 프레임)
while True:
    # 현재 슬라이더 값 읽기
    kp = p.readUserDebugParameter(IDkp)
    kd = p.readUserDebugParameter(IDkd)
    maxForce = p.readUserDebugParameter(IDmaxForce)
    height = p.readUserDebugParameter(IDheight)

    # 읽은 값으로 로봇 제어
    for joint_index in range(12):
        p.setJointMotorControl2(
            robot,
            joint_index,
            p.POSITION_CONTROL,
            targetPosition=target_angles[joint_index],
            force=maxForce,          # ← MaxForce 슬라이더 값 사용
            positionGain=kp,         # ← Kp 슬라이더 값 사용
            velocityGain=kd          # ← Kd 슬라이더 값 사용
        )

    p.stepSimulation()
    time.sleep(1./240.)
```

7.3 슬라이더 값 조정 전략 (PD 게인 튜닝 순서)

1. 초기 설정

- $K_p = 0.012$
- $K_d = 0.4$
- $\text{MaxForce} = 12.5$

2. **Kp** 조정

- Kd를 0으로 설정
- Kp를 천천히 증가
- 진동이 시작하는 지점 확인 (예: $Kp = 0.03$)
- 그보다 약간 낮게 설정 (예: $Kp = 0.025$)

3. **Kd** 조정

- 위에서 찾은 Kp 유지
- Kd를 천천히 증가
- 진동이 빠르게 감소하는 지점 찾기
- 오버슈트가 없는 최소 Kd 값 선택

4. **MaxForce** 조정

- 로봇이 자세를 유지할 수 있는 최소값 찾기
- 너무 높으면 에너지 낭비, 너무 낮으면 중력에 의해 쓰러짐

7.4 실전 예시: 보행 중 파라미터 조정

```
# 초기 설정
IDstepLength = p.addUserDebugParameter("Step Length", -50, 50, 0)
IDstepWidth = p.addUserDebugParameter("Step Width", -30, 30, 0)
IDstepHeight = p.addUserDebugParameter("Step Height", 0, 100, 72)
IDfrequency = p.addUserDebugParameter("Frequency", 0, 5, 1.5)

while True:
    # 슬라이더 값 읽기
    step_length = p.readUserDebugParameter(IDstepLength)
    step_width = p.readUserDebugParameter(IDstepWidth)
    step_height = p.readUserDebugParameter(IDstepHeight)
    frequency = p.readUserDebugParameter(IDfrequency)

    # 보행 패턴에 적용
    foot_positions = calculate_gait(
        time.time(),
        step_length,    # 보폭
        step_width,     # 보폭 너비
        step_height,    # 발 들어올리는 높이
        frequency       # 걷는 속도
    )

    # 역기구학으로 관절 각도 계산 및 적용
    # ...

    p.stepSimulation()
    time.sleep(1./240.)
```

7.5 슬라이더 조합 효과

Kp	Kd	효과
높음	낮음	빠른 응답, 많은 진동, 불안정
높음	높음	빠른 응답, 진동 억제, 안정적
낮음	낮음	느린 응답, 약간 진동, 매우 느림
낮음	높음	느린 응답, 진동 없음, 부드러움

7.6 디버깅용 슬라이더

개발 중 유용한 추가 슬라이더입니다.

```

# 디버그 모드 전환
IDdebug = p.addUserDebugParameter("Debug Mode", 0, 1, 0)

# 시뮬레이션 속도 조절
IDspeed = p.addUserDebugParameter("Sim Speed", 0.1, 2.0, 1.0)

# 특정 다리 선택 (0=전체, 1=앞왼쪽, 2=앞오른쪽...)
IDleg = p.addUserDebugParameter("Select Leg", 0, 4, 0)

# 중력 조절 (다른 행성 시뮬레이션)
IDgravity = p.addUserDebugParameter("Gravity", -20, 0, -9.81)

while True:
    debug_mode = p.readUserDebugParameter(IDdebug) > 0.5
    sim_speed = p.readUserDebugParameter(IDspeed)
    selected_leg = int(p.readUserDebugParameter(IDleg))
    gravity = p.readUserDebugParameter(IDgravity)

    p.setGravity(0, 0, gravity)

    if debug_mode:
        # 디버그 정보 출력
        print(f"Current Kp: {kp}, Kd: {kd}")

    # 시뮬레이션 속도 조절
    time.sleep(1./(240. * sim_speed))

```

7.7 주의사항

1. 슬라이더는 한 번만 생성: `p.addUserDebugParameter()` 는 초기화 시 한 번만 호출
2. 매 프레임 값 읽기: `p.readUserDebugParameter()` 는 루프 안에서 매번 호출
3. 범위 설정 중요: min, max 값을 적절히 설정해야 실험이 용이
4. 기본값 설정: 안전한 기본값으로 시작
5. 실시간 반영: 슬라이더 값은 즉시 시뮬레이션에 반영됨

8. 과제

1. **PD 게인 실험**: 다양한 K_p , K_d 값으로 관절 제어 응답을 관찰한다.
2. **제어 모드 비교**: POSITION, VELOCITY, TORQUE 모드의 차이를 체험한다.
3. **중력 보상**: 중력 보상이 있을 때와 없을 때의 차이를 확인한다.
4. **발끝 위치 제어**: 역기구학을 사용하여 발끝을 원하는 위치로 이동시킨다.

9. 참고 자료

- [PyBullet Quickstart Guide](#)
- [PyBullet API Documentation](#)
- PD Control Theory
- Inverse Dynamics for Gravity Compensation